PeerDrop

Complete Developer Roadmap

A step-by-step guide to building a production-grade peer-to-peer encrypted file transfer platform from scratch

6	20+	15+	100%
Phases	Topics	Side Projects	Free Resources

Built for self-learners — no bootcamp required.

TABLE OF CONTENTS

■ TIP

How to use this guide: Work through each phase in order. Complete at least ONE side project per phase before moving on — that's what cements the knowledge. Don't skip Phase 0.

PHASE 0 — FOUNDATIONS & PREREQUISITES

Before touching WebRTC, make sure these building blocks are solid.

Why this phase?

PeerDrop is a JavaScript-heavy project. If you rush into WebRTC without solid JS fundamentals, you will get lost. This phase takes 3–6 weeks and saves months of confusion later.

What to learn

→ **HTML & CSS Basics**

Structure pages, style components, understand the box model and flexbox.

→ **JavaScript Fundamentals**

Variables, functions, loops, arrays, objects, DOM manipulation.

→ **Async JavaScript**

Promises, async/await, callbacks — critical since WebRTC is async everywhere.

→ **ES6+ Features**

Arrow functions, destructuring, spread/rest, modules (import/export).

→ **Node.js Basics**

Running JS on the server, npm, require vs import, basic http server.

→ **Git & GitHub**

Version control — you will need this to deploy and collaborate.

→ **Basic Networking**

What is a port? IP address? HTTP vs HTTPS? DNS? TCP vs UDP?

Learning Resources

Type	Resource	URL
Video	**The Odin Project (Full Stack)**	https://www.theodinproject.com
Video	**freeCodeCamp JS Course**	https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures/
Docs	**MDN JavaScript Guide**	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
Video	**Traversy Media — Async JS Crash**	https://www.youtube.com/watch?v=PoRJizFvM7s
Video	**Node.js Crash Course — Traversy**	https://www.youtube.com/watch?v=fBNz5xF-Kx4
Book	**Eloquent JavaScript (free online)**	https://eloquentjavascript.net
Video	**Git & GitHub — freeCodeCamp**	https://www.youtube.com/watch?v=RGOj5yH7evk
Course	**CS50 Introduction to CS (free)**	https://cs50.harvard.edu/x

Side Projects for Phase 0

#	Project	What You Build	Skills Gained
1	**Clipboard App**	A single-page app where you type text and it copies to clipboard with a button. Add a counter showing how many times you copied.	DOM manipulation, events, async clipboard API

2	**Local To-Do List**	To-do app that saves tasks to localStorage. Add, delete, mark complete, filter by status.	Arrays, JSON, localStorage, DOM
3	**Fetch Weather App**	Use the free Open-Meteo API to fetch and display weather for any city the user types.	fetch(), async/await, API calls, error handling
4	**Node.js CLI Tool**	Build a command-line tool that reads a text file and counts word frequency.	Node.js, file system module, CLI args

PHASE 1 — WEBRTC CORE — PROOF OF CONCEPT

Get two browser tabs talking directly to each other. No UI needed yet.

The goal of this phase

By the end of Phase 1 you should be able to open two browser tabs, manually paste an SDP offer/answer between them, and successfully transfer a small text message or tiny file. This proves the WebRTC plumbing works before you build anything else on top of it.

Core concepts to master

■ RTCPeerConnection

The central WebRTC object. Manages the connection lifecycle between two peers.

■ SDP — Session Description Protocol

A text blob describing media capabilities. The offer/answer model: peer A creates an offer, peer B responds with an answer.

■ ICE — Interactive Connectivity Establishment

The process of finding a network path between two peers. Generates 'ICE candidates' (possible connection routes).

■ RTCDataChannel

A bidirectional channel for sending arbitrary data (text, ArrayBuffers = file chunks). This is your file pipe.

■ STUN Servers

Help a peer discover its public IP address. Use Google's free STUN: stun:stun.l.google.com:19302

■ ArrayBuffer & Blob

JavaScript types for raw binary data. Files are read as ArrayBuffer, sent as chunks, reassembled into a Blob.

■■ NOTE

The WebRTC handshake order matters: (1) Create PeerConnection, (2) Create DataChannel, (3) Create Offer, (4) Set Local Description, (5) Send offer to peer, (6) Peer sets Remote Description, creates Answer, (7) Exchange ICE candidates. Get this order wrong and nothing connects.

Learning Resources

Type	Resource	URL
Book	**WebRTC for the Curious (free)**	https://webrtcforthecurious.com
Docs	**MDN RTCPeerConnection**	https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection
Codelab	**Google WebRTC Codelab**	https://codelabs.developers.google.com/codelabs/webrtc-web
Video	**WebRTC Crash Course — Hussein Nasser**	https://www.youtube.com/watch?v=FExZvpVvYxA
Docs	**MDN RTCDataChannel**	https://developer.mozilla.org/en-US/docs/Web/API/RTCDataChannel
Library	**simple-peer (abstraction layer)**	https://github.com/feross/simple-peer
Docs	**MDN File API**	https://developer.mozilla.org/en-US/docs/Web/API/File_API

Side Projects for Phase 1

#	Project	What You Build	Skills Gained

1	**Manual WebRTC Chat**	Two tabs. Copy-paste SDP offer/answer manually between them. Send text messages via DataChannel. No server.	RTCPeerConnection, DataChannel, SDP, ICE
2	**Tiny File Transfer**	Extend the chat app to send a small file (<1 MB). Read it as ArrayBuffer, send over DataChannel, reassemble and download on other side.	File API, ArrayBuffer, Blob, DataChannel binary mode
3	**STUN Test Tool**	Build a page that creates a PeerConnection, gathers ICE candidates, and displays your public IP (found in the candidate strings).	ICE gathering, STUN, network concepts

PHASE 2 — SIGNALING SERVER & ROOM SYSTEM

Automate the SDP exchange with a real server. Add rooms, keys and passphrases.

What you are building

A lightweight Node.js server using Socket.io that: generates room keys & passphrases, lets peers join a room, and relays WebRTC signaling messages between them automatically. The server touches zero file bytes — it only passes tiny JSON messages.

Concepts to master

■ WebSockets & Socket.io

Persistent two-way connections between browser and server. Much better than HTTP polling for real-time events.

■ Socket.io Rooms

Built-in Socket.io concept. Clients join a named room; you can emit events to everyone in that room.

■ Signaling Flow

Sender creates offer → emits to room → Receiver sets remote desc, creates answer → emits back → both exchange ICE candidates via the server.

■ Room Key Generation

Use crypto.randomBytes() on the server to generate cryptographically random room keys. Never use Math.random() for this.

■ Express.js Basics

Serve your frontend HTML/JS files and handle simple REST endpoints (create room, validate passphrase).

■ Environment Variables

Store secrets (TURN credentials, DB URLs) in .env files. Use the dotenv package. Never commit secrets to GitHub.

> **■■ NOTE**
> At this stage, add TURN server support even if you don't pay for one yet. Use Metered.ca's free tier (500 MB/month). Without TURN, your app silently fails for ~15% of users behind strict corporate firewalls.

Learning Resources

Type	Resource	URL
Docs	**Socket.io Official Docs**	https://socket.io/docs/v4/
Video	**Socket.io Crash Course — Traversy**	https://www.youtube.com/watch?v=jD7FnbI76Hg
Docs	**Express.js Getting Started**	https://expressjs.com/en/starter/installing.html
Video	**Express.js Crash Course — Traversy**	https://www.youtube.com/watch?v=L72fhGm1tfE
Docs	**Node.js crypto module**	https://nodejs.org/api/crypto.html
Free	**Metered.ca TURN Server (free tier)**	https://www.metered.ca/tools/openrelay/
Docs	**dotenv package**	https://www.npmjs.com/package/dotenv

Side Projects for Phase 2

#	Project	What You Build	Skills Gained

1	**Real-time Chat App**	Multi-room chat app with Socket.io. Users enter a username and room name. Messages broadcast to all room members.	Socket.io rooms, events, broadcasting, Express
2	**Signaling Server**	Build the actual PeerDrop signaling server. Generate room keys, handle join/leave, relay SDP and ICE between exactly two peers.	WebRTC signaling, room management, crypto
3	**Peer Notifier**	Add the 'receiver joined' notification to the sender UI. Show a waiting spinner on sender side until receiver connects.	Socket.io emit/on, UI state management

PHASE 3 — LARGE FILE SUPPORT & FLOW CONTROL

Handle files from 1 MB to 50 GB reliably. Progress bars, pause, resume.

The challenge with large files

WebRTC DataChannel has an internal send buffer (~16 MB). If you blast chunks faster than they can be sent, the buffer fills up and your app crashes or drops data. For large files on the receiver side, you cannot hold everything in RAM — you need to stream directly to disk. This phase is about solving both problems.

Concepts to master

■ Chunking Strategy

Split files using File.slice(start, end) into chunks of 64 KB–256 KB. Send each chunk as an ArrayBuffer. Track chunk index for ordering.

■ Flow Control / Backpressure

Check dataChannel.bufferedAmount before each send. If it exceeds a threshold (~8 MB), pause and wait for the bufferedamountlow event, then resume.

■ File System Access API

Lets the browser write chunks directly to a user-chosen file on disk. Critical for large files. The receiver creates a writable stream and pipes chunks to it.

■ Transfer Metadata Protocol

Before sending file bytes, send a JSON message with: filename, filesize, total chunks, mime type. Receiver uses this to show progress.

■ Progress Tracking

Sender tracks chunks sent. Receiver tracks chunks received. Both sides calculate percentage and transfer speed (bytes / elapsed time).

■ Error Recovery

Handle DataChannel close events mid-transfer. Log the last received chunk index so a future resume can start from there.

■ SHA-256 Checksums

After transfer, both sides compute a checksum of the file using SubtleCrypto.digest(). Compare them to verify the file arrived intact.

■ TIP

Use a chunk size of 64 KB for maximum compatibility. Some browsers or mobile devices struggle with larger chunks. You can increase to 256 KB after detecting a stable connection.

Learning Resources

Type	Resource	URL
Docs	**File System Access API — MDN**	https://developer.mozilla.org/en-US/docs/Web/API/File_System_Access_API
Article	**WebRTC DataChannel Flow Control**	https://developer.mozilla.org/en-US/docs/Web/API/RTCDataChannel/bufferedAmount
Article	**Large File Transfers via DataChannel**	https://webrtc.github.io/samples/src/content/datachannel/filetransfer/
Docs	**SubtleCrypto.digest (SHA-256)**	https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/digest

Sample	**WebRTC File Transfer Sample (Google)**	https://webrtc.github.io/samples/
Article	**Streams API — MDN**	https://developer.mozilla.org/en-US/docs/Web/API/Streams_API

Side Projects for Phase 3

#	Project	What You Build	Skills Gained
1	**1 GB File Transfer Test**	Transfer a 1 GB dummy file between two browser tabs on same machine. Display real-time speed (MB/s) and ETA. Verify checksum after.	Chunking, flow control, progress UI, SubtleCrypto
2	**Chunked Uploader**	Without WebRTC — upload a large file to a Node.js server in chunks using fetch(). Reassemble on server. Shows chunking logic in isolation.	File API, ArrayBuffer, chunking logic, streams
3	**Download Stream Writer**	Build a page where clicking a button generates a 500 MB fake file and saves it to disk chunk-by-chunk using File System Access API.	File System Access API, WritableStream, large data

PHASE 4 — ENCRYPTION & SECURITY LAYER

Add true end-to-end encryption so even your TURN server can't read files.

Why add E2EE on top of WebRTC?

WebRTC already encrypts with DTLS, but when traffic falls back through a TURN relay, the TURN operator can technically read the data. By adding AES-GCM encryption in the browser using the room passphrase as the key seed, you ensure nobody — not even you — can read the file contents. This is the same approach used by Signal and wormhole.app.

Concepts to master

■ Web Crypto API (SubtleCrypto)

Built into every modern browser. No external library needed. Functions: generateKey, deriveKey, encrypt, decrypt, digest.

■ AES-GCM Encryption

AES-256-GCM is authenticated encryption — it encrypts AND verifies integrity in one step. Use it for each chunk.

■ PBKDF2 Key Derivation

Derive a strong 256-bit encryption key from the user's passphrase using PBKDF2 with 100,000 iterations and a random salt.

■ Initialization Vectors (IV)

AES-GCM needs a unique 12-byte IV per chunk. Generate with crypto.getRandomValues(). Send IV alongside each encrypted chunk.

■ Constant-Time Comparison

When validating passphrases on the server, use timing-safe comparison to prevent timing attacks.

■ HTTPS Everywhere

WebRTC and Web Crypto API require a secure context (HTTPS). Local dev can use localhost, but production must have TLS.

■ Content Security Policy

Add CSP headers to your server to prevent XSS attacks that could steal keys from memory.

■ TIP

The encryption flow: passphrase → PBKDF2 (100k iterations, random salt) → AES-256-GCM key → encrypt each chunk with unique IV → send [IV + ciphertext]. Receiver derives same key from same passphrase + salt, decrypts each chunk.

Learning Resources

Type	Resource	URL
Docs	**SubtleCrypto — MDN**	https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto
Article	**Web Crypto API Practical Guide**	https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
Article	**AES-GCM in the Browser**	https://bradyjoslin.com/blog/encryption-webcrypto/
Docs	**PBKDF2 — MDN**	https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/deriveKey
Course	**Cryptography I — Stanford (free)**	https://www.coursera.org/learn/crypto
Article	**HTTPS with Let's Encrypt**	https://letsencrypt.org/getting-started/
Docs	**Content Security Policy — MDN**	https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

Side Projects for Phase 4

#	Project	What You Build	Skills Gained
1	**Browser Encrypt/Decrypt Tool**	A page with two text areas. User types a passphrase and message. Encrypt button shows ciphertext. Decrypt button recovers original. All in browser.	SubtleCrypto, AES-GCM, PBKDF2, key derivation
2	**Encrypted File Locker**	User picks a file and a password. File is encrypted in the browser and downloaded as .enc. A decrypt page recovers the original file.	File API, encryption, IV handling, Blob download
3	**Secure Notes App**	A notes app where notes are encrypted before being saved to localStorage. Passphrase required to read them back.	End-to-end encryption flow, key management

PHASE 5 — AUTH, RESUME & MULTI-RECEIVER (PREMIUM FEATURES)

Build the logged-in user features: accounts, transfer history, resume, and broadcast.

What changes for logged-in users

Authentication unlocks two powerful features that are impossible for anonymous users: (1) Resuming interrupted transfers — the server stores which chunks were received, so a reconnecting peer can continue from where it left off. (2) Sending to multiple receivers simultaneously — each receiver gets their own dedicated WebRTC DataChannel, and all transfers run in parallel.

Concepts to master

■ JWT Authentication

JSON Web Tokens. User logs in, server issues a signed JWT. Client sends it with each request. Stateless — no sessions stored on server.

■ Password Hashing with bcrypt

Never store plain-text passwords. Hash them with bcrypt (cost factor 12). On login, compare submitted password against stored hash.

■ PostgreSQL Basics

Store user accounts and transfer session metadata (not file bytes!). Learn: tables, queries, indexes, foreign keys, basic SQL.

■ Prisma ORM

Type-safe database access for Node.js. Much cleaner than raw SQL for a solo project.

■ Chunk Acknowledgement Protocol

Receiver sends back a 'chunk received' message with the chunk index after each successful chunk. Sender tracks last-acknowledged chunk for resume.

■ Multiple Peer Management

Maintain a Map of peerId → RTCPeerConnection. When sending, iterate the map and send each chunk to every active peer in parallel.

■ Rate Limiting & Abuse Prevention

Limit room creation per IP to prevent abuse. Use the express-rate-limit package.

Learning Resources

Type	Resource	URL
Video	**JWT Auth in Node.js — Traversy**	https://www.youtube.com/watch?v=mbsmsi7l3r4
Docs	**bcrypt.js npm package**	https://www.npmjs.com/package/bcryptjs
Course	**PostgreSQL Tutorial (free)**	https://www.postgresqltutorial.com
Docs	**Prisma Getting Started**	https://www.prisma.io/docs/getting-started
Video	**Full Auth System — Net Ninja**	https://www.youtube.com/watch?v=SnoAwLP1a-0
Docs	**express-rate-limit**	https://www.npmjs.com/package/express-rate-limit
Article	**Refresh Token Rotation**	https://auth0.com/blog/refresh-tokens-what-are-they-and-when-to-use-them/

Side Projects for Phase 5

#	Project	What You Build	Skills Gained

1	**Auth REST API**	Build a full register/login/logout API with Express + PostgreSQL + bcrypt + JWT. Include protected routes that require a valid token.	JWT, bcrypt, SQL, REST API design
2	**Resumable Upload**	A file upload to a Node server that tracks which chunks arrived. If you refresh mid-upload, it resumes from the last chunk.	Chunk acks, server state, resume logic
3	**Multi-Tab Broadcast**	Open 3 browser tabs. Tab 1 is the sender. Tabs 2 and 3 each connect and receive the same file simultaneously over separate WebRTC connections.	Multiple PeerConnections, parallel transfers

PHASE 6 — DEPLOYMENT, SCALING & BEYOND

Ship it. Get PeerDrop live on the internet.

Deployment stack recommendation

Component	Service	Free Tier?
Signaling Server	Railway.app or Render.com	Yes
Frontend	Vercel or Netlify	Yes
TURN Server	Metered.ca (free) → Twilio (paid)	500 MB/mo free
Database	Supabase (PostgreSQL)	Yes
TLS / HTTPS	Let's Encrypt (auto via platforms)	Free forever
Domain	Namecheap / Cloudflare	~$10/year

What to learn for deployment

Type	Resource	URL
Docs	**Railway.app Deployment**	https://docs.railway.app
Docs	**Vercel Deploy Guide**	https://vercel.com/docs
Docs	**Supabase PostgreSQL**	https://supabase.com/docs
Video	**Docker for Beginners — freeCodeCamp**	https://www.youtube.com/watch?v=fqMOX6JJhGo
Docs	**Let's Encrypt Certbot**	https://certbot.eff.org
Article	**WebRTC in Production Checklist**	https://bloggeek.me/webrtc-vs-alternatives/

You've got this.

PeerDrop is a real product — not a tutorial project. The full stack you'll learn (JavaScript, WebRTC, Node.js, Socket.io, PostgreSQL, Web Crypto, React) makes you a genuinely employable full-stack developer. Take it one phase at a time. Build the side projects. Ship the thing.

Start today: open theodinproject.com and complete your first lesson.